

The Software Process

What is a software process?

What are the generic framework activities?

How are processes modeled and what are process patterns?

What are prescriptive process models and their strengths and weaknesses?

What characteristics of incremental models make them amenable to modern software projects?

What is the unified process?

Why is "agility" a watchword in modern software engineering?

What is agile software development and how does it differ from traditional process models?

Software Engineering Definition

1. Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.
2. The study of approaches as in (1).



IEEE Standard 610.12-1990

1

The practical application — applying engineering principles to software development, operation, and maintenance processes in real-world scenarios.

2

The study of approaches as described in definition (1) — the academic and research perspective of software engineering methodologies, processes, and best practices.

Key Characteristics

- ✓ **Systematic:** Organized, methodical approach
- ✓ **Disciplined:** Following established principles
- ✓ **Quantifiable:** Measurable outcomes and quality

Layered Technology

Software engineering is built upon a layered foundation where each layer supports the ones above it.



Tools

Computer-Aided Software Engineering (CASE) tools that automate and support development activities



Methods

Technical "how-to" for building software including communication, analysis, design, construction, and testing



Process Model

Foundation that holds technology together, providing a roadmap for software development



A "Quality" Focus

The bedrock supporting all software engineering — organizational commitment to continuous improvement

Framework Layers

Quality focus forms the foundation. Organizations committed to continuous improvement can build solid process models, effective methods, and integrated tools. Each layer provides essential support for the layers above, creating a robust framework for delivering high-quality software products.

Quality Focus: The Bedrock

Quality is not an add-on; it is the foundation that enables all other software engineering activities.



Bedrock Support

Quality focus is the **bedrock to support software engineering**. Without a solid commitment to quality, even the best processes, methods, and tools will fail to deliver satisfactory results.



Organizational Commitment

It represents an **organizational commitment to software excellence**, where every team member understands that quality is everyone's responsibility, not just the QA department's.



Customer-Centric

Quality focus means understanding and meeting **customer's needs** as the primary driver of all development activities, ensuring delivered software solves real problems.



Continuous Process Improvement

A quality-focused organization embraces **continuous process improvement** as a way of life. This means regularly evaluating current practices, identifying areas for enhancement, implementing changes systematically, and measuring the results to create a virtuous cycle of improvement.

Measurement

Track quality metrics and process performance

Analysis

Identify root causes of quality issues

Implementation

Deploy improved processes systematically

Validation

Verify improvements deliver expected benefits

Process: Definition and Purpose

The process provides the foundation for systematic and rational software development.



Definition

A process is **a collection of activities, actions, and tasks** that are performed when some work product is to be created.
It represents the systematic approach that transforms initial concepts into final deliverables through structured stages.

Three Essential Components



Activities

Broad goals like communication, applicable across projects



Actions

Set of tasks producing a work product (e.g., architecture design)



Tasks

Small, specific goals producing tangible work products



Foundation

The process is the **foundation of software engineering** to hold technology together, enabling timely and rational software construction.



Roadmap

Defines a framework as a **road map for software development**, guiding teams through complex projects from conception to delivery.



Management Control

Forms the **basis for software management control**, enabling tracking, measurement, and corrective actions.

Software Engineering Methods

Methods provide the technical "how-to" for building high-quality software systems.



Technology Provision

Methods provide **the technology: how to build software**. They encompass the techniques, procedures, and heuristics used to create quality software products efficiently and effectively.



Modeling Activity

Methods include **modeling activity and descriptive techniques** that help visualize, specify, construct, and document software systems. Models serve as blueprints for implementation.

Core Method Categories



Communication: Stakeholder interaction and requirements elicitation



Requirements Analysis: Understanding and documenting needs



Design Modeling: Architecture and component design



Program Construction: Coding and implementation



Testing: Verification and validation



Support: Maintenance and evolution

Method Integration

Modern software engineering methods are **integrated and synergistic**. They work together seamlessly:

- Analysis feeds design
- Design guides construction
- Testing validates all phases

Software Engineering Tools

Tools automate and support the methods and processes, enhancing productivity and quality.



For Methods

Tools provide **automated support for methods**, making it easier and faster to apply technical procedures.

- ✓ Design modeling tools
- ✓ Code generation tools
- ✓ Testing frameworks



For Processes

Tools support **process management and control**, helping teams coordinate activities.

- ✓ Project management platforms
- ✓ Version control systems
- ✓ Continuous integration servers



Stakeholder Integration

Tools are **integrated for stakeholder collaboration**, enabling effective communication.

- ✓ Collaboration platforms
- ✓ Requirements management
- ✓ Issue tracking systems



Computer-Aided Software Engineering (CASE)

CASE tools represent the **integrated toolsets** that automate and support the entire software engineering lifecycle. Modern CASE environments provide seamless integration between analysis, design, coding, and testing tools, creating a cohesive development ecosystem that dramatically improves productivity and quality.

Process Framework Overview

A hierarchical structure organizing activities, actions, and tasks.



Umbrella Activities

Span the entire process across all phases

1 Framework Activity #1

Software Engineering Action #1.1

- Work tasks
- Work products
- Milestones & deliverables
- QA checkpoints

Software Engineering Action #1.k ...

n Framework Activity #n

Software Engineering Action #n.1

- Work tasks
- Work products
- Milestones & deliverables
- QA checkpoints

Software Engineering Action #n.m ...

Hierarchical Structure

1

Activities

Umbrella & Framework

2

Actions

Engineering Actions

3

Tasks

Work Tasks

4

Products

Deliverables

Generic Framework Activities

Five fundamental activities form the backbone of every software process.



Communication

Stakeholder interaction: Customer collaboration, requirements gathering, project scope definition, and ongoing communication throughout the project lifecycle.



Planning

Project roadmap: Resource allocation, timeline creation, risk assessment, milestone definition, and project tracking and control activities.



Modeling

Analysis & Design: Requirements analysis, architectural design, component design, data modeling, and creation of system blueprints.



Construction

Code & Testing: Code generation, unit testing, integration testing, debugging, and creation of executable software components.



Deployment

Delivery & Support: Software delivery, user training, installation, maintenance, and ongoing customer support activities.

Activity Characteristics

- ✓ **Sequential:** Generally flow from communication to deployment
- ✓ **Iterative:** Often repeated in cycles for incremental development
- ✓ **Adaptable:** Tailored to project size, complexity, and domain
- ✓ **Universal:** Present in every software development project

Key Insight: While these five activities are **always present**, their implementation varies significantly. A small project might use lightweight versions, while a large, mission-critical system requires rigorous, formal application of each activity.

Process Flow

Different approaches to sequencing framework activities based on project context.

→ Linear

Sequential execution of activities from communication to deployment in a straight line.



Choosing the Right Flow Pattern

Project Factors

- Requirements stability
- Project size and complexity
- Team experience
- Customer involvement

Risk Considerations

- Technical uncertainty
- Market volatility
- Resource availability
- Time-to-market pressure

Organizational Context

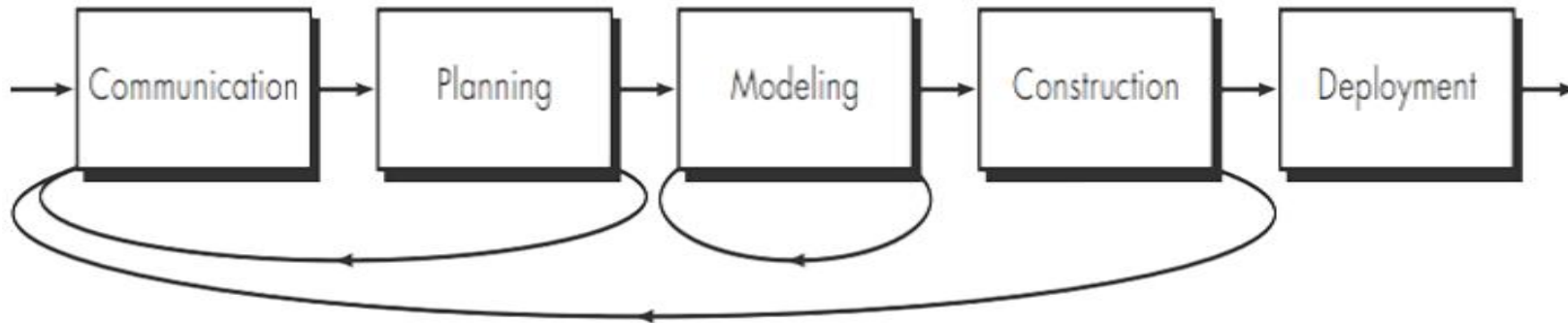
- Organizational maturity
- Available tools and infrastructure
- Regulatory requirements
- Cultural readiness for change

Process Flow

Different approaches to sequencing framework activities based on project context.

Iterative

Cyclical repetition of activities, with each iteration building upon previous work.



Choosing the Right Flow Pattern

Project Factors

- Requirements stability
- Project size and complexity
- Team experience
- Customer involvement

Risk Considerations

- Technical uncertainty
- Market volatility
- Resource availability
- Time-to-market pressure

Organizational Context

- Organizational maturity
- Available tools and infrastructure
- Regulatory requirements
- Cultural readiness for change

Process Flow

Different approaches to sequencing framework activities based on project context.



Evolutionary

Gradually adding features as the product grows over time



Choosing the Right Flow Pattern

Project Factors

- Requirements stability
- Project size and complexity
- Team experience
- Customer involvement

Risk Considerations

- Technical uncertainty
- Market volatility
- Resource availability
- Time-to-market pressure

Organizational Context

- Organizational maturity
- Available tools and infrastructure
- Regulatory requirements
- Cultural readiness for change

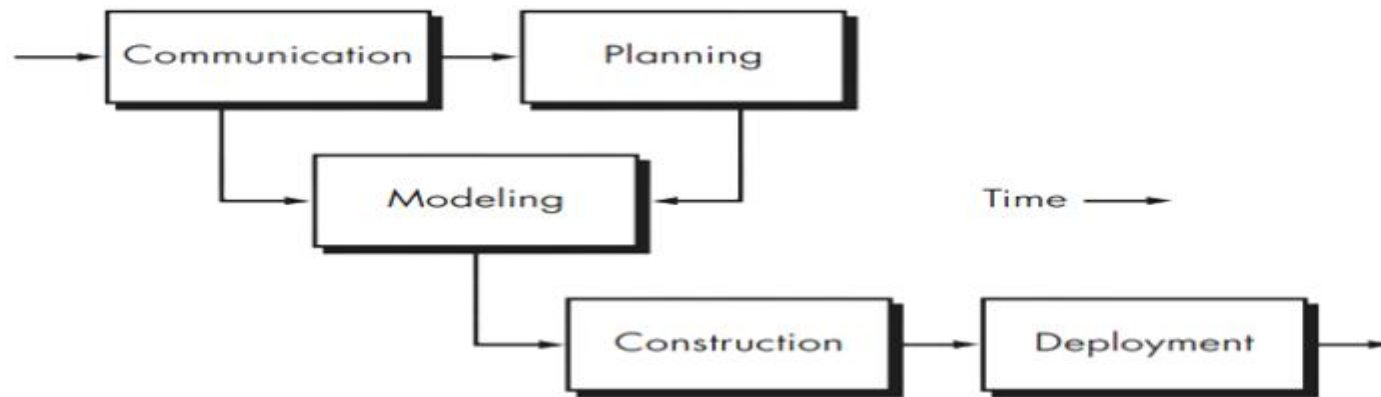
Process Flow

Different approaches to sequencing framework activities based on project context.



Parallel

Working on multiple tasks simultaneously to save time.



Choosing the Right Flow Pattern

Project Factors

- Requirements stability
- Project size and complexity
- Team experience
- Customer involvement

Risk Considerations

- Technical uncertainty
- Market volatility
- Resource availability
- Time-to-market pressure

Organizational Context

- Organizational maturity
- Available tools and infrastructure
- Regulatory requirements
- Cultural readiness for change

Process Flow Characteristics

Each flow pattern has distinct strengths, weaknesses, and optimal application contexts.

Linear (Waterfall)

Strengths

Simple to understand, clear milestones, easy documentation, predictable timeline

Weaknesses

Inflexible to change, late feedback, high risk of building wrong system

Best For

Stable requirements, well-understood domain, fixed contracts

Iterative (Incremental)

Strengths

Early delivery of value, accommodates changing requirements, rapid feedback

Weaknesses

Requires experienced teams, can lead to architectural degradation

Best For

Evolving requirements, complex systems, time-critical projects



Flexibility is Key

Modern software engineering recognizes that **no single process flow is optimal for all projects**. Successful organizations maintain flexibility, selecting the appropriate flow pattern based on project characteristics, team capabilities, customer expectations, and organizational constraints. The most effective processes often combine elements from multiple flow patterns.

Process Flow Characteristics

Each flow pattern has distinct strengths, weaknesses, and optimal application contexts.

Evolutionary (Agile)

Strengths

Highly adaptive, continuous customer feedback, embraces change, rapid delivery

Weaknesses

Less predictable, requires high customer involvement, minimal documentation

Best For

High uncertainty, innovative products, competitive markets

Parallel

Strengths

High throughput, significantly reduced cycle time, efficient resource utilization, better scalability

Weaknesses

High coordination complexity, risk of bottlenecks, potential data/state conflicts, increased overhead

Best For

Independent subtasks, data processing pipelines, high-volume batch operations, multi-perspective analysis



Flexibility is Key

Modern software engineering recognizes that **no single process flow is optimal for all projects**. Successful organizations maintain flexibility, selecting the appropriate flow pattern based on project characteristics, team capabilities, customer expectations, and organizational constraints. The most effective processes often combine elements from multiple flow patterns.

CROSS-CUTTING CONCERNS

Umbrella Activities

Eight critical activities that span the entire software process from inception to completion.



Software Project Management

Planning, tracking, and controlling project activities, resources, and deliverables throughout the lifecycle.



Formal Technical Reviews

Systematic reviews of work products to identify defects, assess quality, and ensure adherence to standards.



Software Quality Assurance

Activities to ensure software meets quality standards through audits, metrics, and process monitoring.



Software Configuration Management

Managing versions, changes, and baselines of software work products throughout development.



Work Product Preparation

Creating, producing, and maintaining documentation, models, and other work products.



Reusability Management

Identifying, creating, cataloging, and reusing software components and patterns to improve productivity.



Measurement

Collecting and analyzing process and product metrics to assess quality, productivity, and progress.



Risk Management

Identifying, analyzing, prioritizing, and mitigating project risks that could impact success.



Umbrella activities are not optional extras. They are essential for professional software development, providing the infrastructure that enables framework activities to succeed. Neglecting these activities significantly increases project risk.

Process Model Adaptability

Framework activities are universal, but their implementation must be adapted to context.



Universal Application

The framework activities will **always be applied on every project**. This is a fundamental principle — you cannot build quality software without communication, planning, modeling, construction, and deployment.



Adaptive Implementation

BUT the tasks and **degree of rigor** for each activity will vary based on multiple contextual factors.



Project Type

A **web application** requires different modeling rigor than an **embedded medical device system**. Safety-critical systems demand extensive formal reviews.



Project Characteristics

Size, complexity, and uncertainty drive adaptation. Large teams need more formal communication. Innovative projects require evolutionary approaches.



Team Judgment

Common sense judgment and **project team concurrence** guide process adaptation. Teams assess what works and adjust accordingly.



The Art of Software Engineering

Effective software engineering requires **intelligent adaptation**. Rigid process application often leads to failure. Successful teams understand that the process serves the project, not vice versa. They apply framework activities thoughtfully, tailoring rigor to maximize value delivery while maintaining quality.

Task Set: Simple Project

Requirements gathering tasks for a small, relatively simple project.



Context

Small, relatively simple project with limited scope, straightforward requirements, and minimal technical risk. Assumes cohesive stakeholder group and established team.

Key Characteristics

Lightweight, informal approach with minimal overhead. Assumes requirements are straightforward and stakeholders can collaborate effectively.

Time Investment

Can be completed in days. Single meeting, simple documentation, and rapid prioritization enable quick project initiation.

Success Factors

Requires stakeholder availability, clear communication, and consensus-building. Works when scope is well-bounded.

Task Set: Requirements Gathering

- 1 **Make a list of stakeholders** — Identify all individuals and groups with interest in the project
- 2 **Invite stakeholders to informal meeting** — Single gathering for collaborative discussion
- 3 **Ask for features and functions** — Each stakeholder creates list of desired capabilities
- 4 **Discuss and build final list** — Group discussion to consolidate and clarify requirements
- 5 **Prioritize requirements** — Rank by importance, value, and urgency
- 6 **Note areas of uncertainty** — Document ambiguous items needing clarification

Task Set: Complex Project

Requirements gathering for a large, more complex project with multiple stakeholder groups.



Context

Large, complex project with diverse stakeholder groups, conflicting requirements, significant technical risk, and need for formal documentation and traceability.

- 1 **Make stakeholder list** — Comprehensive identification including primary, secondary, and tertiary stakeholders
- 2 **Interview stakeholders separately** — One-on-one meetings to understand individual perspectives
- 3 **Build preliminary function list** — Consolidate input into structured feature inventory
- 4 **Schedule facilitated meetings** — Structured workshops with trained facilitator
- 5 **Conduct meetings & produce scenarios** — Facilitated sessions creating user stories

6

Refine scenarios based on feedback — Iterate user stories with stakeholder input

7

Build revised stakeholder requirements — Formal requirements specification document

8

Use QFD to prioritize — Quality Function Deployment matrix for requirement ranking

9

Package for incremental delivery — Group requirements into logical, releasable increments

10

Note constraints and restrictions — Document technical, business, and regulatory constraints

11

Discuss validation methods — Define acceptance criteria and verification approaches

Additional Techniques

QFD, use-case modeling, facilitated meetings, stakeholder analysis matrices, and formal traceability.

Time Investment

Requires weeks to months. Multiple iterations, extensive documentation, and stakeholder alignment take time.

Success Factors

Requires skilled facilitator, executive sponsorship, stakeholder commitment, and formal change control.

Essence of Practice

Four fundamental steps that form the foundation of all engineering problem-solving.

1

Understand the Problem

Comprehend requirements and constraints. Identify stakeholders, clarify needs, define success criteria, and understand the problem domain thoroughly.

Key Questions: Who are the stakeholders? What are their needs? What is the business context? What constraints exist?

2

Plan the Solution

Design approach and architecture. Create system models, define components, select technologies, and establish development strategy.

Key Activities: Architectural design, technology selection, resource planning, risk assessment, timeline creation

3

Carry Out the Plan

Implement and execute. Write code, create documentation, conduct testing, manage project execution, and track progress against the plan.

Key Activities: Coding, testing, integration, documentation, project tracking, stakeholder communication

4

Examine the Result

Verify and validate. Conduct reviews, perform testing, measure against requirements, gather feedback, and assess whether the solution solves the original problem.

Key Activities: Verification, validation, user acceptance testing, quality assurance, lessons learned



Iterative by Nature

These four steps are not linear but form a continuous cycle. Examination often reveals new understanding, requiring replanning and additional implementation. Modern software engineering embraces this iteration, delivering value incrementally through multiple cycles rather than attempting a perfect single pass.

General Principles of Software Engineering

Timeless principles that guide effective software development practice.



The Reason It All Exists

Software is built to **provide value to users**. Technical elegance is secondary. Every decision should prioritize delivering tangible business and user value.



Maintain the Vision

A **clear vision** of the end product is essential. It guides decisions, maintains team alignment, and ensures the final product achieves its intended purpose.



Be Open to the Future

Software must **evolve gracefully**. Design for change, use modular architectures, and avoid technologies that lock you into a single path.



KISS (Keep It Simple, Stupid)

Simplicity is the ultimate sophistication. Simple solutions are easier to understand, test, maintain, and evolve. Avoid unnecessary complexity.



What You Produce, Others Will Consume

Software is built for **stakeholders and users**. Design with their needs, capabilities, and contexts in mind. Consider long-term maintainability.



Plan Ahead for Reuse

Reusable assets dramatically improve productivity and quality. Design for reuse, create shared libraries, and invest in component repositories.



Think!

Above all, think critically. Apply experience, judgment, and creativity. Question assumptions, evaluate alternatives, and make informed decisions.

Process Patterns: Definition and Types

Patterns capture proven solutions to recurring problems in software processes.



Definition

Process patterns **define a set of activities, actions, work tasks, work products and/or related behaviors** that have been proven effective in specific contexts. They capture reusable knowledge from successful projects.

Pattern Template

A template is used to define a pattern, providing a consistent structure for documenting reusable process knowledge.

- ✓ Pattern Name and Intent
- ✓ Type and Initial Context
- ✓ Problem and Solution
- ✓ Resulting Context
- ✓ Related Patterns
- ✓ Known Uses and Examples

Process Pattern Types



Task Patterns

Define small, specific objectives that produce tangible work products (e.g., requirements gathering)



Stage Patterns

Define activities comprising multiple related tasks that accomplish a major goal (e.g., communication)



Phase Patterns

Define high-level lifecycle phases spanning multiple stages (e.g., inception, elaboration)

Customer Communication (Process Activity)

Analysis (Action)

Requirements Gathering (Task)

Design Model (Work Product)

Process Pattern Template

Standardized structure for documenting reusable process patterns.



Pattern Name

Descriptive name that captures the essence of the pattern (e.g., "Customer Communication Pattern")



Intent

The goal or objective this pattern aims to achieve — what problem it solves



Type

Classification as Task Pattern, Stage Pattern, or Phase Pattern



Initial Context

Conditions and prerequisites that must be satisfied before applying this pattern



Problem

The specific challenge or set of challenges this pattern addresses



Solution

Description of how to solve the problem, including activities, tasks, and work products



Resulting Context

The state after successfully applying the pattern — what has been accomplished

Additional Elements

Related Patterns: Other patterns that work well with this one

Known Uses/Examples: Real-world applications of this pattern

Standardization Benefits: Using a consistent template ensures patterns are comprehensive, understandable, and comparable. This enables organizations to build libraries of proven practices that can be selected and applied to new projects.

COMPLETE EXAMPLE

Process Pattern Example: Communication

A comprehensive stage pattern for establishing effective customer collaboration.

Pattern Name	Communication
Intent	To establish a collaborative relationship with the customer in an effort to define project scope, business requirements and other project constraints.
Type	Stage pattern
Initial Context	(1) Appropriate stakeholders have been identified and are willing to participate in communication (2) Stakeholders agree that a problem exists and that software may provide a solution
Problem	Requirements must be elicited from stakeholders and organized in a way that can be used by software engineers. All stakeholders must collaborate to define requirements and to identify those areas where requirements are uncertain.
Solution	Each stakeholder must develop a description of the functions, features, information and behavior that are exhibited by the software. To accomplish this, a structured, facilitated meeting is conducted.
Resulting Context	When this pattern has been successfully completed, basic information required for the development of an analysis model has been acquired and documented in some manner. Use-cases (user scenarios) have been developed, along with basic descriptions of system function and behavior and the data objects that are to be manipulated and/or produced.
Related Pattern	Conducted a meeting; requirement gathering; developing use-cases; building a mini-spec; negotiating requirements, prioritization.
Known Uses/Example	Communication is mandatory at the beginning of every software project; is recommended throughout the software project; and is mandatory once the deployment activity is underway.

Software Process Assessment

Industry standards for evaluating and improving software development maturity.



ISO Standards

International Organization for Standardization

provides globally recognized standards for software process assessment and quality management.

- ✓ ISO/IEC 12207: Software lifecycle processes
- ✓ ISO/IEC 15504: Process assessment (SPICE)
- ✓ ISO 9001: Quality management systems



SEI: CMM

Software Engineering Institute's **Capability Maturity Model** provides a framework for process improvement with five maturity levels.

- ✓ Five maturity levels from Initial to Optimizing
- ✓ Key Process Areas (KPAs) for each level
- ✓ Focus on software-specific processes



CMMI

Capability Maturity Model Integration unifies multiple CMMs into a comprehensive framework covering systems, software, and integrated product development.

- ✓ Integrates software, systems, and IPD models
- ✓ Two representations: Staged and Continuous
- ✓ Process areas organized by maturity levels



Why Assessment Matters

Process assessment provides **objective evaluation** of current practices, identifies improvement opportunities, enables benchmarking against industry standards, and creates a roadmap for systematic process enhancement. Organizations with higher maturity levels consistently deliver higher quality software with greater predictability and efficiency.

CMM: Capability Maturity Model

A framework for process improvement of software development.

Level	Characteristics	Management visibility	Prediction of performance
Optimized 5	The management focuses on the continuous improvement of the process		
Managed 4	Product and process are managed quantitatively		
Defined 3	The software is managed in accordance with a well-defined process		
Repeatable 2	A project management system is in place; broad visibility over the process exists (Milestones).		
Initial 1	No visibility inside the development process		

Level	Key Process Areas	Quality and productivity
5	Defects prevention Technology change management Process change management	
4	Quantitative process management Software quality management	
3	Peer reviews Inter-group coordination Software product engineering Integrated software management Training program Organization process definition Organization process focus	
2	Software project planning Software project tracking & oversight Software configuration management Software subcontract management Requirements management Software quality assurance	
1	Successes depends on individuals !	

CMMI: Capability Maturity Model Integration

A unified framework for process improvement across software, systems, and integrated product development.



What is CMMI?

CMMI is a **comprehensive framework for process improvement** that provides organizations with the essential elements of effective processes. It integrates multiple capability maturity models into a unified whole.

Integration Benefits

- ✓ Unifies software and systems engineering
- ✓ Covers entire product lifecycle
- ✓ Provides both staged and continuous models
- ✓ Enables benchmarking and comparison

CMMI Architecture



Process Areas

Categories of related practices



Goals

Desired outcomes



Practices

Recommended activities



Maturity Levels

Process sophistication



Representations

Staged & Continuous

Systems Engineering

Development of total systems

Software Engineering

Application of engineering to software

Integrated Product Development

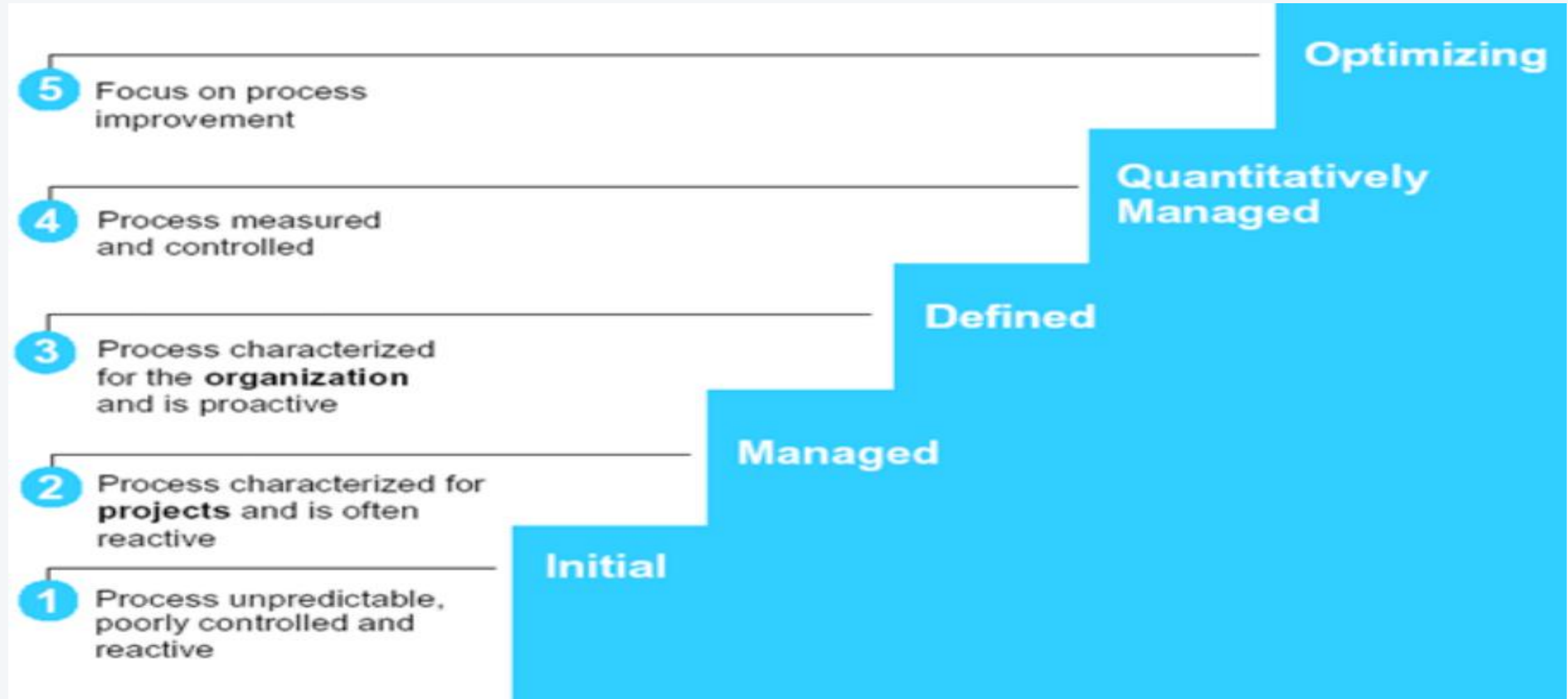
Concurrent engineering approaches

Supplier Sourcing

Acquisition and supply processes

CMMI Maturity Levels

Five levels representing increasing process sophistication and capability.



Summary: Software Engineering Process

Key concepts integrating process, methods, and tools for software development excellence.

1

Software Engineering Discipline

SE is a discipline that integrates **process, methods, and tools** for software development. It applies systematic, disciplined, and quantifiable engineering approaches to create high-quality software products that deliver value to stakeholders.

2

Process Models

Different process models propose varying approaches, but all define **framework activities, tasks, work products, and umbrella activities**. The five generic framework activities — Communication, Planning, Modeling, Construction, and Deployment — provide a universal foundation adapted to project context.

3

Process Patterns

Process patterns define characteristics of proven processes through templates including **intent, context, problem, solution, and resulting context**. They enable organizations to capture and reuse successful practices across projects.

4

CMMI Framework

CMMI is a comprehensive process meta-model describing **specific goals, practices, and capabilities** for mature software processes. Its five maturity levels provide a roadmap for systematic process improvement, enabling organizations to evolve from ad-hoc practices to optimized, continuously improving processes.



Understanding these concepts enables software engineers to select, adapt, and apply appropriate processes for their specific project context, leading to more predictable outcomes, higher quality products, and greater stakeholder satisfaction.

Exercise Set 1

Test your understanding of software engineering fundamentals.

1. Which of the items listed below is not one of the software engineering layers? (Answer: b)
 - a. Process
 - b. Manufacturing
 - c. Methods
 - d. Tools

2. Software engineering umbrella activities are only applied during the initial phases of software development projects.(Answer: b)
 - a. True
 - b. False

3. Which of these are the 5 generic software engineering framework activities? (Answer: a)
 - a. communication, planning, modeling, construction, deployment
 - b. communication, risk management, measurement, production, reviewing
 - c. analysis, designing, programming, debugging, maintenance
 - d. analysis, planning, designing, programming, testing

4. Which of these terms are level names in the Capability Maturity Model? (Answer: e)
 - a. Performed
 - b. Repeatable
 - c. Reused
 - d. Optimized
 - e. Both b and d

Exercise Set 2

Test your understanding of software engineering fundamentals.

5. Which of the items listed below is not one of the process patterns. (Answer: d)
 - a. Intent
 - b. Solution
 - c. Resulting Context
 - d. Output
6. Process technology tools allow software organizations to compress schedules by skipping unimportant activities. (Answer: b)
 - a. True
 - b. False
7. It is generally accepted that one cannot have weak software processes and create high quality end products. (Answer: a)
 - a. True
 - b. False
8. The tasks (and degree of rigor) for each activity are always unchanged. (Answer: b)
 - a. True
 - b. False